

ISA, Microarchitecture, Single-Cycle MIPS, and Dataflow

CS214 – Computer Organization

Where Are We in the Course?

Already discussed

- Digital logic and sequential circuits
- FSMs and control
- ISA concepts and LC-3
- LC-3 multi-cycle microarchitecture

Today

- ISA vs. microarchitecture
- Why ISAs evolve slowly
- x86 → ARM example: Apple silicon
- Build a MIPS single-cycle datapath
- Performance: latency, clock period, CPI
- Single-cycle vs. multi-cycle
- Beyond von Neumann: dataflow

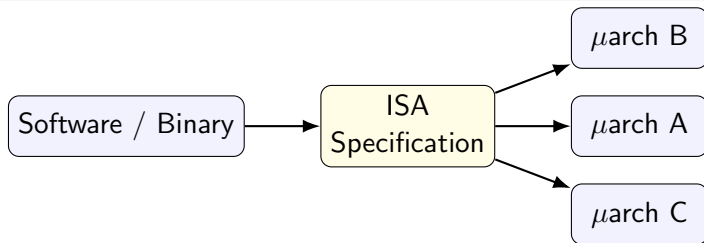
ISA vs. Microarchitecture

Instruction Set Architecture (ISA)

The **contract visible to software**: instructions, opcodes, registers, data types, addressing modes, memory/exception behavior, etc.

Microarchitecture (μ arch)

The **hardware implementation** of that ISA under design goals such as performance, power, cost, and complexity.



What Belongs Where?

ISA	Microarchitecture
ADD opcode / encoding	Ripple-carry vs. carry-lookahead adder
Number of architectural registers	Number of register-file ports
Program counter exists and its visible semantics	How PC update hardware is organized
Memory-visible behavior	Cache levels, size, associativity
Instruction semantics	Single-cycle / multi-cycle / pipelined
Architectural exceptions	Speculation / out-of-order mechanisms

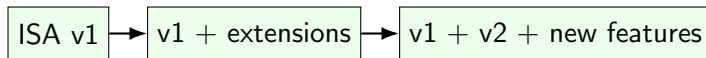
Key test

If software must know it to generate or interpret a binary correctly, it is usually part of the ISA. If hardware can change it transparently, it is usually microarchitecture.

How Does an ISA Change Over Time?

An ISA can evolve by adding or revising architectural features:

- new instructions or instruction-set extensions,
- wider registers / address space,
- new vector, cryptographic, or floating-point operations,
- new privilege, virtualization, memory-ordering, or security features,
- deprecating old features (usually slowly).



Important

Microarchitecture normally changes **much faster** than ISA. A new processor generation can keep the same ISA while changing pipelines, caches, execution units, prediction, and scheduling.

Why Are General-Purpose ISAs Usually Backward Compatible?

- Existing software is already distributed as binaries for an ISA.
- Users expect old applications and operating-system components to keep working.
- Breaking the ISA creates a large software migration cost.
- Therefore, vendors often **extend** an ISA rather than replace it.

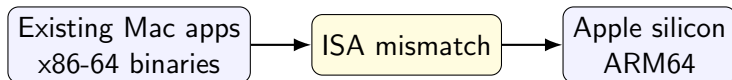


But backward compatibility is a design choice, not a law

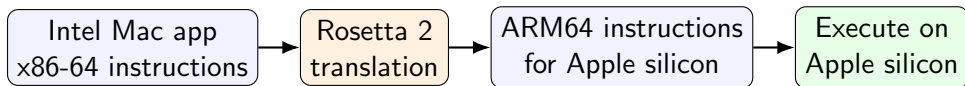
A platform can move to a different ISA if the expected benefits justify the software transition cost.

Example: Apple Moves Mac from x86-64 to ARM64

- Intel-based Macs executed **x86-64** machine code.
- Apple announced the Mac transition to **Apple silicon** in 2020.
- Native Apple-silicon Mac applications use the **arm64** architecture.
- Motivation included a common architecture across Apple products and strong performance per watt.



Rosetta 2: Preserving Software Compatibility Across ISAs



- Rosetta translates x86-64 code so it can execute on Apple silicon.
- Apple documents both **just-in-time (JIT)** and **ahead-of-time (AOT)** translation paths.
- A Universal binary can contain both x86-64 and arm64 code; the native slice is preferred.

Conceptual lesson

Backward compatibility can be provided by the **same ISA in hardware** or, during a transition, by a **software/hardware translation layer**.

Transition: We Built a Multi-Cycle LC-3. What Next?

LC-3 multi-cycle idea

Instruction processing is divided into states/cycles. Intermediate programmer-invisible state (e.g., IR, MAR, MDR) holds partial results while an FSM sequences the datapath.

Now ask a different design question

Can we design a processor in which an entire instruction completes in **one clock cycle**?

Multi-cycle LC-3 $\longrightarrow$ Single-cycle MIPS

MIPS Instructions We Will Support

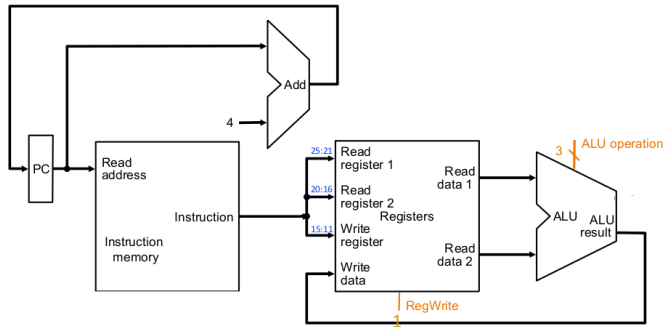
Instruction	Meaning	Format
add \$d,\$s,\$t	$R[d] \leftarrow R[s] + R[t]$	R-type
addi \$t,\$s,imm	$R[t] \leftarrow R[s] + \text{SEXT}(imm)$	I-type
lw \$t,off(\$s)	$R[t] \leftarrow M[R[s] + \text{SEXT}(off)]$	I-type
sw \$t,off(\$s)	$M[R[s] + \text{SEXT}(off)] \leftarrow R[t]$	I-type

32-bit MIPS fields

R-type: op[31:26] rs[25:21] rt[20:16] rd[15:11] shamt[10:6] funct[5:0]

I-type: op[31:26] rs[25:21] rt[20:16] immediate[15:0]

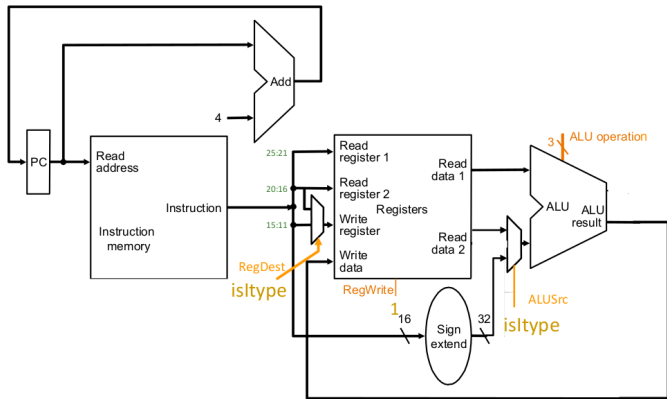
Step 1: Single-Cycle Datapath for add



Example: add \$t0,\$t1,\$t2

- 1 PC addresses instruction memory; instruction is fetched.
- 2 rs and rt select two source registers.
- 3 Register file outputs $R[rs]$ and $R[rt]$.
- 4 ALU performs ADD.
- 5 ALU result is written into rd; $\text{RegWrite} = 1$.
- 6 In parallel, PC is updated to $\text{PC}+4$.

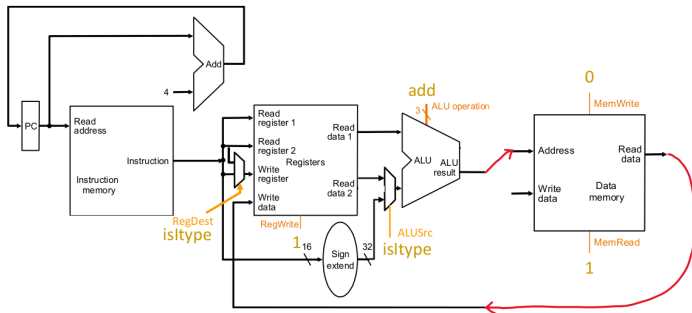
Step 2: Extend the Datapath for addi



Example: `addi $t0,$t1,5`

- 1 Fetch instruction and read source register `rs`.
- 2 Sign-extend the 16-bit immediate to 32 bits.
- 3 `ALUSrc` selects the immediate instead of `Read data 2`.
- 4 ALU computes $R[rs] + SEXT(imm)$.
- 5 `RegDst` selects `rt` as destination.
- 6 Result is written to `rt`; `RegWrite = 1`.

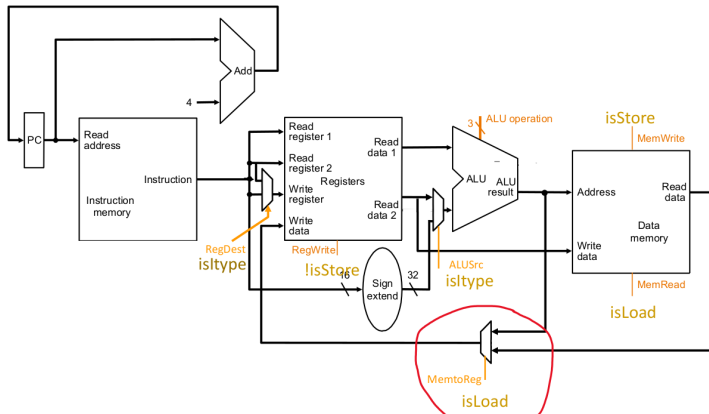
Step 3: Extend the Datapath for 1w



Example: `lw $t0,8($t1)`

- 1 Read base register `rs` and sign-extend offset.
- 2 ALU adds $R[rs] + SEXT(offset)$ to form effective address.
- 3 Data memory is read at this address; `MemRead` = 1.
- 4 `MemtoReg` selects memory data, not the ALU result.
- 5 `RegDst` selects `rt`; `RegWrite` stores loaded word in `rt`.

Load and Store: Data-Memory Path



Common step: ALU computes $R[rs] + \text{SEXT}(\text{offset})$. **Load (lw):**

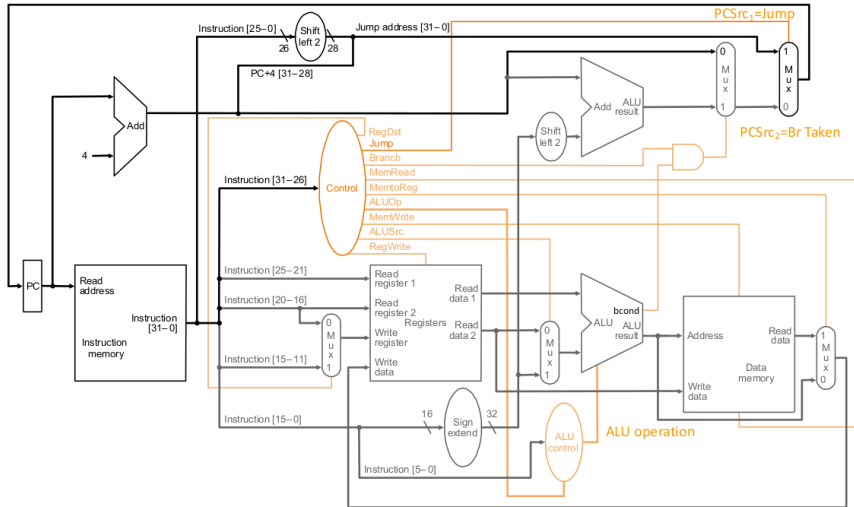
- $\text{MemRead} = 1, \text{MemWrite} = 0$.
- Memory output returns through *MemtoReg* and is written to *rt*.

Store (sw):

- *rt* supplies the value to memory *Write data*.
- $\text{MemWrite} = 1, \text{RegWrite} = 0$; no register is written.

Why two memories? Instruction fetch and data-memory access must be possible simultaneously in the same cycle.

Full Single-Cycle MIPS Datapath (Including Branch and Jump)



Control Signals for Our Instructions

	RegDst	ALUSrc	MemRead	MemWrite	MemtoReg	RegWrite
add	1	0	0	0	0	1
addi	0	1	0	0	0	1
lw	0	1	1	0	1	1
sw	X	1	0	1	X	0

Why is the ALU used for lw and sw?

The ALU computes the effective memory address:

$$R[rs] + \text{SEXT}(\text{offset}).$$

Why is MemtoReg needed?

For lw, the register write-data comes from **data memory** rather than the ALU result. For sw, no register is written.

Simplified Component Delays

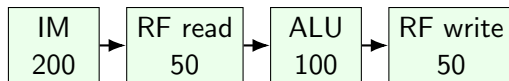
Following the lecture example, assume:

Operation / resource	Delay
Memory access	200 ps
Register-file read	50 ps
ALU operation	100 ps
Register-file write	50 ps

Latency of add

add: instruction fetch → register read → ALU → register write

$$T_{add} = 200 + 50 + 100 + 50 = \boxed{400 \text{ ps}}$$



But in a single-cycle processor...

add does *not* get a 400 ps clock. Every instruction shares one common clock period.

Latency of addi

addi: instruction fetch $\rightarrow$ register read $\rightarrow$ ALU $\rightarrow$ register write

$$T_{addi} = 200 + 50 + 100 + 50 = \boxed{400 \text{ ps}}$$

The sign-extension and MUX selection are treated as negligible in this simplified example.

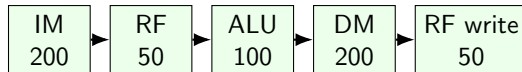
Observation

Although addi uses a different operand source and destination field than add, the dominant resources in this timing model are the same.

Latency of lw : The Critical Path

lw : instruction fetch $\rightarrow$ register read $\rightarrow$ address ALU $\rightarrow$ data memory $\rightarrow$ register write

$$T_{lw} = 200 + 50 + 100 + 200 + 50 = \boxed{600 \text{ ps}}$$



Critical-path consequence

If lw is the slowest supported instruction, the single-cycle clock period must be at least 600 ps.

Clock Period and Clock Frequency

For the single-cycle processor:

$$T_{clk} \geq \max(T_{all \text{ instructions}}) = 600 \text{ ps}$$

Therefore,

$$f_{max} = \frac{1}{T_{clk}} = \frac{1}{600 \times 10^{-12}} \approx \boxed{1.67 \text{ GHz}}$$

Important consequence

Even though add needs only about 400 ps of useful datapath delay, it still occupies the full 600 ps clock cycle.

Processor Performance Formula

For one instruction:

$$T_{instruction} = CPI \times T_{clk}$$

For a program with N instructions:

$$T_{program} = N \times CPI_{avg} \times T_{clk}$$

Since $f = 1/T_{clk}$:

$$T_{program} = \frac{N \times CPI_{avg}}{f}$$

Single-cycle

$CPI = 1$, but T_{clk} is long because it is set by the slowest instruction.

Single-Cycle vs. Multi-Cycle Microarchitecture

	Single-cycle	Multi-cycle
Instruction	1 clock cycle	Multiple cycles
CPI	Exactly 1	Depends on instruction
Clock period	Set by slowest instruction	Set by slowest stage/state
Hardware reuse	Limited; more parallel resources	High; ALU/memory can be reused
Intermediate state	No instruction-internal intermediate registers are required	IR/MDR/A/B/ALUOut etc.
Control	Simpler combinational control	FSM / sequenced control

Advantages and Disadvantages

Single-cycle

- + Simple conceptual model
- + $CPI = 1$
- Slowest instruction sets clock period
- Fast instructions waste cycle time
- Often duplicates hardware/resources

Multi-cycle

- + Shorter clock period
- + Simple instructions can use fewer cycles
- + Reuse expensive hardware
- $CPI > 1$ and varies
- Intermediate registers + sequencing overhead
- More complex control

A Subtle Point: Which One Is Faster?

You cannot decide from CPI alone or clock frequency alone.

Performance depends on $CPI_{avg} \times T_{clk}$

Example:

- Single-cycle: $CPI = 1$, $T_{clk} = 600 \text{ ps} \Rightarrow 600 \text{ ps/instruction}$.
- Hypothetical multi-cycle: $CPI_{avg} = 4$, $T_{clk} = 200 \text{ ps} \Rightarrow 800 \text{ ps/instruction}$.

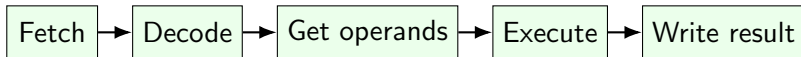
In this hypothetical case, single-cycle is faster despite the lower frequency. A different stage partition or instruction mix can reverse the result.

Always compare total execution time

$N \times CPI_{avg} \times T_{clk}$, not just GHz.

So Far: The von Neumann Execution View

Our LC-3 and MIPS discussions use the familiar instruction-driven model:



- A program is represented as a sequence of instructions.
- A program counter identifies the next instruction to fetch.
- Control directs datapath actions.

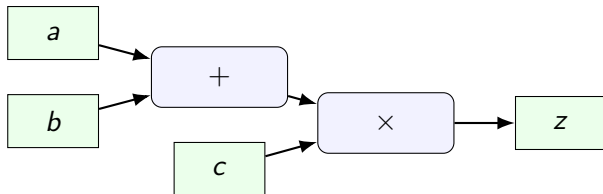
Question

Must computation always be driven by the next instruction selected by a PC?

Another Model: Dataflow

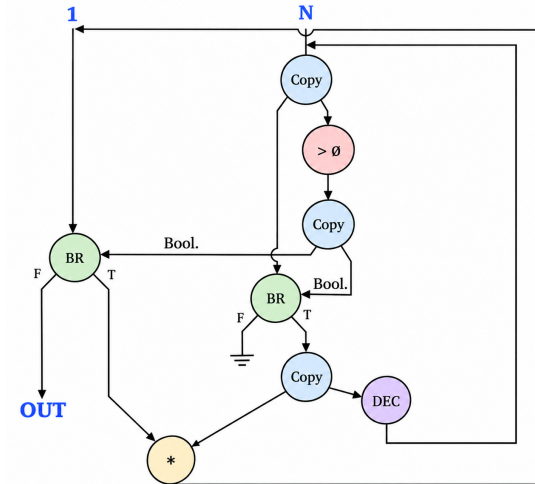
Core idea

An operation becomes eligible to execute when its **required input operands are available**, rather than simply because a program counter reached it.



For $z = (a + b) \times c$, the add fires when a, b are ready; the multiply fires when $(a + b)$ and c are ready.

Dataflow Example: Factorial



Operations execute when their required input tokens are available. BR routes tokens using the Boolean condition, DEC updates N , and $*$ generates the partial product.

Control Flow vs. Dataflow

Control-flow / von Neumann style

- PC chooses next instruction
- Explicit instruction order
- Dependencies constrain execution
- Natural model for general-purpose software

Dataflow style

- Operand availability enables operation
- Independent operations can execute concurrently
- Dependencies are explicit graph edges
- Natural for exposing parallelism

Do not overstate the distinction

Modern processors are not usually “pure dataflow computers.” They often expose a conventional ISA while using dataflow-like mechanisms internally.

Why Is the Dataflow Idea Useful?

- **Parallelism is explicit:** independent graph nodes can run together.
- **No need to serialize independent work** merely because instructions appear in sequence.
- Maps naturally to workloads with many repeated numerical operations.
- Useful when computation can be represented as producer-consumer dependencies.

But why did pure dataflow not replace von Neumann machines?

General-purpose programs need efficient handling of memory, control flow, exceptions, precise state, synchronization, and compatibility. Pure dataflow machines introduce substantial token/scheduling/communication complexity.

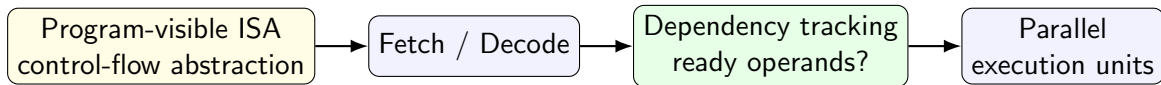
Where Do We See Dataflow Principles Today?

- **Out-of-order CPUs:** instructions can issue when operands and execution resources are ready, while the ISA remains conventional.
- **GPUs:** massive parallel execution and dependency scheduling exploit data-parallel workloads (not a pure dataflow ISA).
- **FPGAs / spatial accelerators:** pipelines can be physically organized around producer-consumer data movement.
- **Systolic arrays / ML accelerators:** data flows rhythmically through arrays of processing elements, enabling high reuse and throughput.
- **Streaming / signal-processing hardware:** natural fit when data items continuously move through a computation graph.

Modern design pattern

Conventional ISA outside, dataflow-like execution inside is common because it combines software compatibility with hardware parallelism.

Connecting the Models



Key architectural insight

The **ISA is an abstraction**. A processor can preserve sequential architectural behavior while its microarchitecture executes work internally in a much more parallel, data-driven manner.

Takeaways

- 1 ISA specifies *what*; microarchitecture decides *how*.
- 2 ISA compatibility matters because binaries depend on the ISA contract.
- 3 Apple changed Mac ISA from x86-64 to ARM64 and used Rosetta 2 to bridge legacy software.
- 4 A MIPS single-cycle datapath can be built incrementally: `add` $\rightarrow$ `addi` $\rightarrow$ `lw` $\rightarrow$ `sw` $\rightarrow$ full datapath.
- 5 In single-cycle design, the slowest instruction determines the clock period.
- 6 Compare processors using $T = N \times CPI_{avg} \times T_{clk}$.
- 7 Dataflow is an important alternative execution model and, today, a powerful principle inside many parallel processors and accelerators.

- Harris & Harris, *Digital Design and Computer Architecture*, Ch. 7.1–7.4 .
- Patterson & Hennessy, *Computer Organization and Design*, MIPS datapath material .
- Apple, *Apple announces Mac transition to Apple silicon*, 2020.
- Apple Developer / Platform Security documentation, *Rosetta translation environment / Rosetta 2 on a Mac with Apple silicon*.